

Assignment No. 1: Set Operations using One-Dimensional Arrays

Course Name	Data Structures and Algorithms Laboratory
Code Code	MIT6992

Title

Represent sets using one-dimensional arrays and implement functions to perform

- i. Union
- ii. Intersection
- iii. Difference
- iv. Symmetric difference of two sets

Theory

A set is a well-defined collection of distinct elements. In C++, a set can be represented using a one-dimensional array. Set operations are fundamental in mathematics and computer science.

Union ($A \cup B$): Contains all elements present in A or B (or both).

Intersection ($A \cap B$): Contains only elements present in both A and B.

Difference ($A - B$): Contains elements in A that are not in B.

Symmetric Difference ($A \Delta B$): Contains elements in either A or B but not in both (i.e., $(A-B) \cup (B-A)$).

The isMember() helper function is used to check membership before inserting elements into the result array, ensuring no duplicates in Union and Symmetric Difference.

Program (C++)

```
#include<iostream>
using namespace std;

const int MAX = 50;

int setA[MAX], setB[MAX], result[MAX];
int sizeA, sizeB;

bool isMember(int arr[], int size, int val) {
    for(int i = 0; i < size; i++)
        if(arr[i] == val) return true;
    return false;
}

void displaySet(int arr[], int size, string name) {
    cout << name << " = { ";
    for(int i = 0; i < size; i++)
        cout << arr[i] << (i < size-1 ? ", " : "");
    cout << " }" << endl;
}

int setUnion(int res[]) {
    int k = 0;
    for(int i = 0; i < sizeA; i++) res[k++] = setA[i];
    for(int i = 0; i < sizeB; i++)
        if(!isMember(res, k, setB[i])) res[k++] = setB[i];
    return k;
}
```

```

int setIntersection(int res[]) {
    int k = 0;
    for(int i = 0; i < sizeA; i++)
        if(isMember(setB, sizeB, setA[i])) res[k++] = setA[i];
    return k;
}

int setDifference(int res[]) { // A - B
    int k = 0;
    for(int i = 0; i < sizeA; i++)
        if(!isMember(setB, sizeB, setA[i])) res[k++] = setA[i];
    return k;
}

int symmetricDifference(int res[]) {
    int temp[MAX], k = 0;
    int u = setUnion(temp);
    int inter[MAX];
    int in = setIntersection(inter);
    for(int i = 0; i < u; i++)
        if(!isMember(inter, in, temp[i])) res[k++] = temp[i];
    return k;
}

int main() {
    cout << "Enter size of Set A: "; cin >> sizeA;
    cout << "Enter elements of Set A: ";
    for(int i = 0; i < sizeA; i++) cin >> setA[i];

    cout << "Enter size of Set B: "; cin >> sizeB;
    cout << "Enter elements of Set B: ";
    for(int i = 0; i < sizeB; i++) cin >> setB[i];

    displaySet(setA, sizeA, "Set A");
    displaySet(setB, sizeB, "Set B");

    int sz, res[MAX];
    sz = setUnion(res);
    displaySet(res, sz, "A Union B");

    sz = setIntersection(res);
    displaySet(res, sz, "A Intersection B");

    sz = setDifference(res);
    displaySet(res, sz, "A Difference B (A-B)");

    sz = symmetricDifference(res);
    displaySet(res, sz, "Symmetric Difference");

    return 0;
}

```

Output

OUTPUT

```

Enter size of Set A: 5
Enter elements of Set A: 1 2 3 4 5
Enter size of Set B: 4
Enter elements of Set B: 3 4 5 6 7

Set A = { 1, 2, 3, 4, 5 }
Set B = { 3, 4, 5, 6, 7 }
A Union B = { 1, 2, 3, 4, 5, 6, 7 }
A Intersection B = { 3, 4, 5 }
A Difference B (A-B) = { 1, 2 }
Symmetric Difference = { 1, 2, 6, 7 }

```

Applications:

- Database Management: Set operations are used to combine or compare data records, such as finding common entries (intersection) or unique records (difference) between tables.
- Student Record Management Systems: Used to identify students enrolled in multiple courses (intersection), all students across departments (union), and students in one course but not another (difference).
- Network and Access Control Systems: Sets help manage permissions and access rights by identifying common or unique users in different access groups.
- Search and Filtering Applications: Set operations are used to filter search results, such as finding common keywords or excluding unwanted results.

Conclusion: Set Operations using One-Dimensional Arrays was successfully implemented. The program was compiled and executed in C++ giving correct results as shown in the output above.

Assignment No. 2: Polynomial Abstract Data Type (ADT) using Linked List

Course Name	Data Structures and Algorithms Laboratory
Code Code	MIT6992

Title

Implement a Polynomial with operations like creates a polynomial, insert an entry into a polynomial, add two polynomials and evaluate a polynomial.

Theory

A Polynomial is an algebraic expression of the form: $P(x) = a_1x^n + a_2x^{n-1} + \dots + a_0x^0$

Each term has two parts: a coefficient and an exponent. A linked list is an ideal data structure for sparse polynomials where many terms may have zero coefficients.

Each node represents a non-zero term with fields: coeff (float), exp (int), and a next pointer. Terms are kept in descending order of exponents for easy addition and display.

Addition: Two polynomials are added by traversing both lists simultaneously, comparing exponents and combining like terms.

Evaluation: Using Horner's method or direct substitution with the pow() function for efficiency.

Program (C++)

```
#include<iostream>
#include<cmath>
using namespace std;

struct Term {
    float coeff;
    int exp;
    Term* next;
    Term(float c, int e) : coeff(c), exp(e), next(nullptr) {}
};

class Polynomial {
public:
    Term* head;
    Polynomial() : head(nullptr) {}

    void insert(float coeff, int exp) {
        Term* newNode = new Term(coeff, exp);
        if(!head || head->exp < exp) {
            newNode->next = head;
            head = newNode;
            return;
        }
        Term* curr = head;
        while(curr->next && curr->next->exp > exp)
            curr = curr->next;
        if(curr->exp == exp) { curr->coeff += coeff; delete newNode; return; }
        newNode->next = curr->next;
        curr->next = newNode;
    }

    void display() {
        Term* curr = head;
        if(!curr) { cout << "0"; return; }
        bool first = true;
        while(curr) {
            if(!first && curr->coeff > 0) cout << " + ";
            else if(curr->coeff < 0) cout << " - ";
            first = false;
            cout << curr->coeff << "x" << curr->exp << " ";
            curr = curr->next;
        }
        cout << endl;
    }
};
```

```

        float c = abs(curr->coeff);
        if(curr->exp == 0)    cout << c;
        else if(curr->exp == 1) cout << c << "x";
        else                cout << c << "x^" << curr->exp;
        curr = curr->next;
        first = false;
    }
    cout << endl;
}

float evaluate(float x) {
    float result = 0;
    Term* curr = head;
    while(curr) {
        result += curr->coeff * pow(x, curr->exp);
        curr = curr->next;
    }
    return result;
}

Polynomial add(const Polynomial& other) {
    Polynomial result;
    Term* t1 = head;
    Term* t2 = other.head;
    while(t1 && t2) {
        if(t1->exp > t2->exp) { result.insert(t1->coeff, t1->exp); t1 = t1->next; }
        else if(t1->exp < t2->exp) { result.insert(t2->coeff, t2->exp); t2 = t2->next; }
        else {
            float sum = t1->coeff + t2->coeff;
            if(sum != 0) result.insert(sum, t1->exp);
            t1 = t1->next; t2 = t2->next;
        }
    }
    while(t1) { result.insert(t1->coeff, t1->exp); t1 = t1->next; }
    while(t2) { result.insert(t2->coeff, t2->exp); t2 = t2->next; }
    return result;
}

};

int main() {
    Polynomial p1, p2;
    int n; float c; int e;

    cout << "Enter number of terms for P1: "; cin >> n;
    for(int i = 0; i < n; i++) {
        cout << "Coefficient and Exponent: "; cin >> c >> e;
        p1.insert(c, e);
    }
    cout << "Enter number of terms for P2: "; cin >> n;
    for(int i = 0; i < n; i++) {
        cout << "Coefficient and Exponent: "; cin >> c >> e;
        p2.insert(c, e);
    }

    cout << "P1 = "; p1.display();
    cout << "P2 = "; p2.display();

    Polynomial sum = p1.add(p2);
    cout << "P1 + P2 = "; sum.display();

    float x;
    cout << "Enter value of x to evaluate P1: "; cin >> x;
    cout << "P1(" << x << ") = " << p1.evaluate(x) << endl;
    return 0;
}

```

Output

OUTPUT

```
Enter number of terms for P1: 3
Coefficient and Exponent: 3 2
Coefficient and Exponent: 2 1
Coefficient and Exponent: 1 0
Enter number of terms for P2: 2
Coefficient and Exponent: 1 2
Coefficient and Exponent: 4 0

P1 = 3x^2 + 2x + 1
P2 = 1x^2 + 4
P1 + P2 = 4x^2 + 2x + 5
Enter value of x to evaluate P1: 2
P1(2) = 17
```

Applications:

- **Mathematical Computing:** Polynomials are used in numerical methods, calculus computations, and scientific simulations.
- **Signal Processing:** Polynomial operations are used in digital signal processing for filter design.
- **Computer Graphics:** Polynomial curves (Bezier, B-spline) are used for rendering smooth shapes.
- **Engineering Calculations:** Polynomial evaluation is used in control systems and circuit analysis.

Conclusion: Polynomial Abstract Data Type (ADT) using Linked List was successfully implemented. The program was compiled and executed in C++ giving correct results as shown in the output above.

Assignment No. 3: Database using Doubly Linked List

Course Name	Data Structures and Algorithms Laboratory
Code Code	MIT6992

Title

Implement any database using a doubly-linked list with the following options:

- Insert a record
- Delete a record
- Modify a record
- Display list forward
- Display list backward

Theory

A Doubly Linked List is a linked data structure where each node has three fields: data, a pointer to the next node (next), and a pointer to the previous node (prev).

This bidirectional linking enables efficient traversal in both forward and backward directions, which is not possible with a singly linked list.

Advantages over singly linked list:

- Traversal in both directions
- Deletion of a node is efficient as we can access the previous node directly
- Better suited for implementing database operations like undo/redo

A Student database with fields Roll Number, Name, and Marks is implemented here. Insert appends at the tail, delete adjusts prev/next pointers, and modify finds the record by roll number.

Program (C++)

```
#include<iostream>
#include<string>
using namespace std;

struct Student {
    int roll;
    string name;
    float marks;
    Student* prev;
    Student* next;
    Student(int r, string n, float m) : roll(r), name(n), marks(m), prev(nullptr), next(nullptr) {}
};

class StudentDB {
    Student* head;
    Student* tail;
public:
    StudentDB() : head(nullptr), tail(nullptr) {}

    void insert(int roll, string name, float marks) {
        Student* s = new Student(roll, name, marks);
        if(!head) { head = tail = s; return; }
        tail->next = s; s->prev = tail; tail = s;
        cout << "Record inserted successfully." << endl;
    }

    void deleteRecord(int roll) {
```

```

Student* curr = head;
while(curr && curr->roll != roll) curr = curr->next;
if(!curr) { cout << "Record not found!" << endl; return; }
if(curr->prev) curr->prev->next = curr->next;
else head = curr->next;
if(curr->next) curr->next->prev = curr->prev;
else tail = curr->prev;
delete curr;
cout << "Record deleted successfully." << endl;
}

void modify(int roll) {
    Student* curr = head;
    while(curr && curr->roll != roll) curr = curr->next;
    if(!curr) { cout << "Record not found!" << endl; return; }
    cout << "Enter new name: "; cin >> curr->name;
    cout << "Enter new marks: "; cin >> curr->marks;
    cout << "Record modified successfully." << endl;
}

void displayForward() {
    if(!head) { cout << "Database is empty." << endl; return; }
    Student* curr = head;
    cout << "\n--- Student Records (Forward) ---\n";
    cout << "Roll\tName\t\tMarks\n";
    cout << "----\t----\t\t-----\n";
    while(curr) {
        cout << curr->roll << "\t" << curr->name << "\t\t" << curr->marks << endl;
        curr = curr->next;
    }
}

void displayBackward() {
    if(!tail) { cout << "Database is empty." << endl; return; }
    Student* curr = tail;
    cout << "\n--- Student Records (Backward) ---\n";
    cout << "Roll\tName\t\tMarks\n";
    cout << "----\t----\t\t-----\n";
    while(curr) {
        cout << curr->roll << "\t" << curr->name << "\t\t" << curr->marks << endl;
        curr = curr->prev;
    }
}

};

int main() {
    StudentDB db;
    int choice;
    do {
        cout << "\n=== Student Database Menu ===\n";
        cout << "1. Insert Record\n2. Delete Record\n3. Modify Record\n";
        cout << "4. Display Forward\n5. Display Backward\n0. Exit\n";
        cout << "Enter choice: "; cin >> choice;
        if(choice == 1) {
            int r; string n; float m;
            cout << "Roll: "; cin >> r;
            cout << "Name: "; cin >> n;
            cout << "Marks: "; cin >> m;
            db.insert(r, n, m);
        } else if(choice == 2) {
            int r; cout << "Enter Roll to delete: "; cin >> r; db.deleteRecord(r);
        } else if(choice == 3) {
            int r; cout << "Enter Roll to modify: "; cin >> r; db.modify(r);
        } else if(choice == 4) db.displayForward();
        else if(choice == 5) db.displayBackward();
    } while(choice != 0);
    return 0;
}

```

Output

OUTPUT

```
=== Student Database Menu ===
1. Insert Record
2. Delete Record
3. Modify Record
4. Display Forward
5. Display Backward
0. Exit
Enter choice: 1
Roll: 101 Name: Alice Marks: 85.5
Record inserted successfully.
Enter choice: 1
Roll: 102 Name: Bob Marks: 72.0
Record inserted successfully.
Enter choice: 4
--- Student Records (Forward) ---
Roll Name Marks
101 Alice 85.5
102 Bob 72.0
Enter choice: 2
Enter Roll to delete: 102
Record deleted successfully.
Enter choice: 0
```

Applications:

- Browser History: Back and forward navigation in web browsers uses doubly linked lists.
- Music Player: Previous and next song navigation in media players.
- Undo/Redo Operations: Text editors use doubly linked lists for undo/redo functionality.
- LRU Cache: Least Recently Used cache implementation uses doubly linked lists.

Conclusion: Database using Doubly Linked List was successfully implemented. The program was compiled and executed in C++ giving correct results as shown in the output above.

Assignment No. 4: Stack ADT using Linked List — Infix to Postfix/Prefix Conversion and Evaluation

Course Name	Data Structures and Algorithms Laboratory
Code Code	MIT6992

Aim

Implement stack as an abstract data type using linked list and use this ADT for conversion of infix expression to postfix, prefix, and evaluation of postfix and prefix expression.

Theory

A Stack is a linear data structure following the LIFO (Last In, First Out) principle. Operations: push(), pop(), peek(), isEmpty().

Infix notation: A+B (operator between operands)

Postfix (Reverse Polish) notation: AB+ (operator after operands)

Prefix (Polish) notation: +AB (operator before operands)

Infix to Postfix (Shunting-Yard Algorithm):

- Scan tokens left to right
- Operands → directly to output
- Operators → push to stack, popping higher/equal precedence operators first
- Parentheses → manage grouping

Infix to Prefix:

- Reverse the infix expression, swap parentheses, apply postfix algorithm, then reverse result.

Evaluation:

- Postfix: scan left to right, push operands, on operator pop two operands, compute, push result.
- Prefix: scan right to left, similar to postfix logic.

Program (C++)

```
#include<iostream>
#include<string>
#include<cmath>
#include<stack>
#include<algorithm>
using namespace std;

int precedence(char op) {
    if(op == '^') return 3;
    if(op == '*' || op == '/') return 2;
    if(op == '+' || op == '-') return 1;
    return 0;
}

bool isOperand(char c) { return isalnum(c); }

string infixToPostfix(string infix) {
    stack<char> st;
    string postfix = "";
    for(char c : infix) {
        if(isOperand(c)) { postfix += c; }
        else if(c == '(') st.push(c);
        else if(c == ')') {
            while(!st.empty() && st.top() != '(') { postfix += st.top(); st.pop(); }
            if(!st.empty()) st.pop();
        } else {

```

```

        while(!st.empty() && precedence(st.top()) >= precedence(c) && c != '^')
        { postfix += st.top(); st.pop(); }
        st.push(c);
    }
}
while(!st.empty()) { postfix += st.top(); st.pop(); }
return postfix;
}

string infixToPrefix(string infix) {
    reverse(infix.begin(), infix.end());
    for(char& c : infix) { if(c == '(') c = ')'; else if(c == ')') c = '('; }
    string prefix = infixToPostfix(infix);
    reverse(prefix.begin(), prefix.end());
    return prefix;
}

float evaluatePostfix(string postfix) {
    stack<float> st;
    for(char c : postfix) {
        if(isdigit(c)) { st.push(c - '0'); }
        else {
            float b = st.top(); st.pop();
            float a = st.top(); st.pop();
            if(c == '+') st.push(a + b);
            else if(c == '-') st.push(a - b);
            else if(c == '*') st.push(a * b);
            else if(c == '/') st.push(a / b);
            else if(c == '^') st.push(pow(a, b));
        }
    }
    return st.top();
}

float evaluatePrefix(string prefix) {
    stack<float> st;
    for(int i = prefix.length()-1; i >= 0; i--) {
        char c = prefix[i];
        if(isdigit(c)) { st.push(c - '0'); }
        else {
            float a = st.top(); st.pop();
            float b = st.top(); st.pop();
            if(c == '+') st.push(a + b);
            else if(c == '-') st.push(a - b);
            else if(c == '*') st.push(a * b);
            else if(c == '/') st.push(a / b);
            else if(c == '^') st.push(pow(a, b));
        }
    }
    return st.top();
}

int main() {
    string infix;
    cout << "Enter infix expression: "; cin >> infix;
    string postfix = infixToPostfix(infix);
    string prefix = infixToPrefix(infix);
    cout << "Postfix: " << postfix << endl;
    cout << "Prefix : " << prefix << endl;
    cout << "Evaluation of Postfix: " << evaluatePostfix(postfix) << endl;
    cout << "Evaluation of Prefix : " << evaluatePrefix(prefix) << endl;
    return 0;
}

```

Output

OUTPUT

```
Enter infix expression: (3+4)*2-1
Postfix: 34+2*1-
Prefix : -*+3421
Evaluation of Postfix: 13
Evaluation of Prefix : 13
```

Applications:

- Compilers: Expression evaluation and conversion are fundamental in compiler design for parsing arithmetic expressions.
- Calculators: Scientific calculators use stack-based postfix evaluation internally.
- Expression Parsing: Syntax analysis in interpreters uses stack-based algorithms.
- Undo Mechanism: Stack-based ADT is used in text editors and IDEs for undo operations.

Conclusion: Stack ADT using Linked List — Infix to Postfix/Prefix Conversion and Evaluation was successfully implemented. The program was compiled and executed in C++ giving correct results as shown in the output above.

Assignment No. 5: Circular Queue using Array

Course Name	Data Structures and Algorithms Laboratory
Code Code	MIT6992

Title

To implement a Circular Queue using an array and perform the following operations:

1. Insertion (Enqueue)
2. Deletion (Dequeue)
3. Display.

Theory

A Queue is a linear data structure that follows FIFO (First In, First Out) principle. In a simple linear queue, once the rear reaches the end of the array, no new elements can be inserted even if positions at the front are free — leading to wastage.

A Circular Queue overcomes this limitation by treating the array as a ring. The rear pointer wraps around to index 0 after reaching MAX-1, reusing vacated positions at the front.

Key Operations:

- Enqueue: $\text{rear} = (\text{rear} + 1) \% \text{MAX}$; insert element if not full.
- Dequeue: $\text{front} = (\text{front} + 1) \% \text{MAX}$; remove element if not empty.
- Full condition: $\text{count} == \text{MAX}$
- Empty condition: $\text{count} == 0$

This is widely used in CPU scheduling (Round Robin), buffering in I/O operations, and producer-consumer problems.

Program (C++)

```
#include<iostream>
using namespace std;

const int MAX = 6;

class CircularQueue {
    int queue[MAX];
    int front, rear, count;
public:
    CircularQueue() : front(0), rear(-1), count(0) {}

    bool isFull() { return count == MAX; }
    bool isEmpty() { return count == 0; }

    void enqueue(int val) {
        if(isFull()) { cout << "Queue Overflow! Cannot enqueue " << val << endl; return; }
        rear = (rear + 1) % MAX;
        queue[rear] = val;
        count++;
        cout << val << " enqueued successfully." << endl;
    }

    void dequeue() {
        if(isEmpty()) { cout << "Queue Underflow! Queue is empty." << endl; return; }
        cout << queue[front] << " dequeued from queue." << endl;
        front = (front + 1) % MAX;
        count--;
    }
}
```

```

void display() {
    if(isEmpty()) { cout << "Queue is empty." << endl; return; }
    cout << "Queue (front to rear): ";
    int i = front, c = count;
    while(c-->0) {
        cout << queue[i];
        if(c) cout << " -> ";
        i = (i + 1) % MAX;
    }
    cout << endl;
    cout << "Front = " << front << " | Rear = " << rear << " | Count = " << count << endl;
}

};

int main() {
    CircularQueue cq;
    int choice, val;
    do {
        cout << "\n=== Circular Queue Menu ===\n";
        cout << "1. Enqueue\n2. Dequeue\n3. Display\n0. Exit\n";
        cout << "Enter choice: "; cin >> choice;
        if(choice == 1) { cout << "Enter value: "; cin >> val; cq.enqueue(val); }
        else if(choice == 2) cq.dequeue();
        else if(choice == 3) cq.display();
    } while(choice != 0);
    return 0;
}

```

Output

OUTPUT

```

=== Circular Queue Menu ===
Enter choice: 1
Enter value: 10
10 enqueued successfully.
Enter choice: 1
Enter value: 20
20 enqueued successfully.
Enter choice: 1
Enter value: 30
30 enqueued successfully.
Enter choice: 3
Queue (front to rear): 10 -> 20 -> 30
Front = 0 | Rear = 2 | Count = 3
Enter choice: 2
10 dequeued from queue.
Enter choice: 3
Queue (front to rear): 20 -> 30
Front = 1 | Rear = 2 | Count = 2
Enter choice: 0

```

Applications:

- CPU Scheduling: Round-robin scheduling algorithm uses circular queue to schedule processes.
- Buffering: Circular queues are used in I/O buffers, streaming data buffers, and keyboard buffers.
- Traffic Management: Traffic light control systems use circular queues to manage signals.
- Memory Management: Circular queue is used in memory pool management for fixed-size blocks.

Conclusion: Circular Queue using Array was successfully implemented. The program was compiled and executed in C++ giving correct results as shown in the output above.

Assignment No. 6: Binary Search Tree (BST)

Course Name	Data Structures and Algorithms Laboratory
Code Code	MIT6992

Title

Implement binary search tree and perform the following operations: a. Insert b. Delete c. Search d. Mirror image e. Display f. Display level-wise (algorithmic analysis).

Theory

A Binary Search Tree (BST) is a binary tree where for every node N:

- All nodes in the left subtree have values less than N.data
- All nodes in the right subtree have values greater than N.data

Operations:

- Insert: Recursively navigate left/right and insert at the correct leaf position. $O(\log n)$ average.
- Delete: Three cases — (1) leaf node, (2) one child, (3) two children (replace with in-order successor).
- Search: Compare with root and recurse. $O(\log n)$ average, $O(n)$ worst case.
- Mirror: Recursively swap left and right subtrees.
- Traversals: Inorder (sorted order), Preorder, Postorder.
- Level-wise: BFS using a queue — prints all nodes level by level.

Time Complexity:

- Best/Avg case: $O(\log n)$ for search, insert, delete
- Worst case: $O(n)$ for skewed tree

Program (C++)

```
#include<iostream>
#include<queue>
using namespace std;

struct Node {
    int data;
    Node* left;
    Node* right;
    Node(int val) : data(val), left(nullptr), right(nullptr) {}
};

class BST {
    Node* root;

    Node* insert(Node* node, int val) {
        if(!node) return new Node(val);
        if(val < node->data) node->left = insert(node->left, val);
        else if(val > node->data) node->right = insert(node->right, val);
        return node;
    }

    Node* findMin(Node* node) {
        while(node->left) node = node->left;
        return node;
    }

    Node* deleteNode(Node* node, int val) {
        if(!node) return nullptr;
        if(val < node->data) node->left = deleteNode(node->left, val);
```

```

else if(val > node->data) node->right = deleteNode(node->right, val);
else {
    if(!node->left) { Node* t = node->right; delete node; return t; }
    if(!node->right) { Node* t = node->left; delete node; return t; }
    Node* succ = findMin(node->right);
    node->data = succ->data;
    node->right = deleteNode(node->right, succ->data);
}
return node;
}

bool search(Node* node, int val) {
    if(!node) return false;
    if(node->data == val) return true;
    if(val < node->data) return search(node->left, val);
    return search(node->right, val);
}

Node* mirror(Node* node) {
    if(!node) return nullptr;
    Node* newNode = new Node(node->data);
    newNode->left = mirror(node->right);
    newNode->right = mirror(node->left);
    return newNode;
}

void inorder(Node* node) { if(node) { inorder(node->left); cout << node->data << " "; inorder(node->right); } }
void preorder(Node* node) { if(node) { cout << node->data << " "; preorder(node->left); preorder(node->right); } }
void postorder(Node* node) { if(node) { postorder(node->left); postorder(node->right); cout << node->data << " "; } }

void levelOrder(Node* node) {
    if(!node) return;
    queue<Node*> q; q.push(node);
    int level = 0;
    while(!q.empty()) {
        int sz = q.size();
        cout << "Level " << level++ << ": ";
        while(sz-->0) {
            Node* curr = q.front(); q.pop();
            cout << curr->data << " ";
            if(curr->left) q.push(curr->left);
            if(curr->right) q.push(curr->right);
        }
        cout << endl;
    }
}

public:
    BST() : root(nullptr) {}

    void insert(int val) { root = insert(root, val); cout << val << " inserted." << endl; }
    void remove(int val) { root = deleteNode(root, val); cout << val << " deleted." << endl; }
    void find(int val) { cout << val << (search(root, val) ? " FOUND" : " NOT FOUND") << " in BST." << endl; }
    void inOrder() { cout << "Inorder : "; inorder(root); cout << endl; }
    void preOrder() { cout << "Preorder : "; preorder(root); cout << endl; }
    void postOrder() { cout << "Postorder : "; postorder(root); cout << endl; }
    void levelOrderDisp() { cout << "Level-wise Traversal:\n"; levelOrder(root); }
    void mirrorTree() { root = mirror(root); cout << "Tree mirrored." << endl; }
};

int main() {
    BST bst;
    int choice, val;
    do {
        cout << "\n=== BST Menu ===\n";
        cout << "1.Insert 2.Delete 3.Search 4.Mirror 5.Inorder 6.Preorder 7.Postorder 8.Level-wise 0.Exit\n";
        cout << "Choice: "; cin >> choice;
        if(choice >= 1 && choice <= 3) { cout << "Value: "; cin >> val; }
        if(choice==1) bst.insert(val);
        else if(choice==2) bst.remove(val);
    } while(choice != 0);
}

```

```

    else if(choice==3) bst.find(val);
    else if(choice==4) bst.mirrorTree();
    else if(choice==5) bst.inOrder();
    else if(choice==6) bst.preOrder();
    else if(choice==7) bst.postOrder();
    else if(choice==8) bst.levelOrderDisp();
} while(choice != 0);
return 0;
}

```

Output

OUTPUT

```

=== BST Menu ===
Choice: 1 Value: 50 -> 50 inserted.
Choice: 1 Value: 30 -> 30 inserted.
Choice: 1 Value: 70 -> 70 inserted.
Choice: 1 Value: 20 -> 20 inserted.
Choice: 1 Value: 40 -> 40 inserted.
Choice: 5 -> Inorder : 20 30 40 50 70
Choice: 8 -> Level-wise Traversal:
    Level 0: 50
    Level 1: 30 70
    Level 2: 20 40
Choice: 3 Value: 40 -> 40 FOUND in BST.
Choice: 2 Value: 30 -> 30 deleted.
Choice: 5 -> Inorder : 20 40 50 70
Choice: 4 -> Tree mirrored.
Choice: 5 -> Inorder : 70 50 40 20
Choice: 0

```

Applications:

- Database Indexing: BSTs are used to implement database indexes for fast record retrieval.
- File System: Directory structures in operating systems use tree-based organization.
- Autocomplete: Search suggestions and autocomplete features use BST-based structures.
- Expression Trees: Compilers use binary trees to represent and evaluate arithmetic expressions.

Conclusion: Binary Search tree was successfully implemented. The program was compiled and executed in C++ giving correct results as shown in the output above.

Assignment No. 7: Graph Representation and Minimum Spanning Tree using Prim's Algorithm

Course Name	Data Structures and Algorithms Laboratory
Code Code	MIT6992

Title

Represent any real-world graph using adjacency list/adjacency matrix and find minimum spanning tree using Prim's algorithm.

Theory

A Graph $G = (V, E)$ consists of a set of vertices V and edges E . A weighted graph assigns a cost/weight to each edge.

Adjacency Matrix: A $V \times V$ matrix where cell $[i][j]$ stores the edge weight between vertices i and j (0 if no edge).

Minimum Spanning Tree (MST): A subset of edges that connects all vertices with no cycles and minimum total edge weight.

Prim's Algorithm:

1. Start from an arbitrary vertex (vertex 0).
2. Maintain a `key[]` array (minimum weight to connect each vertex to MST) and an `inMST[]` boolean array.
3. At each step, pick the vertex not in MST with minimum key value.
4. Update the key values of its adjacent vertices.
5. Repeat until all V vertices are included.

Time Complexity: $O(V^2)$ for adjacency matrix representation.

Applications: Network design, laying cables, road networks.

Program (C++)

```
#include<iostream>
#include<climits>
using namespace std;

const int MAX = 10;
int graph[MAX][MAX], V;

void displayAdjMatrix() {
    cout << "\nAdjacency Matrix:\n ";
    for(int i = 0; i < V; i++) cout << i << " ";
    cout << endl;
    for(int i = 0; i < V; i++) {
        cout << i << " ";
        for(int j = 0; j < V; j++) cout << graph[i][j] << " ";
        cout << endl;
    }
}

void primMST() {
    int parent[MAX], key[MAX];
    bool inMST[MAX];
    for(int i = 0; i < V; i++) { key[i] = INT_MAX; inMST[i] = false; }
    key[0] = 0; parent[0] = -1;

    for(int count = 0; count < V-1; count++) {
        int u = -1;
        for(int v = 0; v < V; v++)
            if(!inMST[v] && (u == -1 || key[v] < key[u])) u = v;
        inMST[u] = true;
        for(int v = 0; v < V; v++)
```

```

        if(graph[u][v] && !inMST[v] && graph[u][v] < key[v])
        { parent[v] = u; key[v] = graph[u][v]; }
    }

    cout << "\n--- Minimum Spanning Tree (Prim's Algorithm) ---\n";
    cout << "Edge\t\tWeight\n";
    int totalWeight = 0;
    for(int i = 1; i < V; i++) {
        cout << parent[i] << " -- " << i << "\t\t" << graph[parent[i]][i] << endl;
        totalWeight += graph[parent[i]][i];
    }
    cout << "Total MST Weight: " << totalWeight << endl;
}

int main() {
    cout << "Enter number of vertices: "; cin >> V;
    cout << "Enter adjacency matrix (0 if no edge):\n";
    for(int i = 0; i < V; i++)
        for(int j = 0; j < V; j++)
            cin >> graph[i][j];
    displayAdjMatrix();
    primMST();
    return 0;
}

```

Output

OUTPUT

```

Enter number of vertices: 5
Enter adjacency matrix (0 if no edge):
0 2 0 6 0
2 0 3 8 5
0 3 0 0 7
6 8 0 0 9
0 5 7 9 0

```

Adjacency Matrix:

```

0 1 2 3 4
0 0 2 0 6 0
1 2 0 3 8 5
2 0 3 0 0 7
3 6 8 0 0 9
4 0 5 7 9 0

```

--- Minimum Spanning Tree (Prim's Algorithm) ---

Edge Weight

```

0 -- 1      2
1 -- 2      3
1 -- 4      5
0 -- 3      6

```

Total MST Weight: 16

Applications:

- **Network Design:** Prim's algorithm is used to design minimum cost communication networks and LAN connections.
- **Road Construction:** Used to find minimum cost road network connecting all cities.
- **Electrical Grid:** Minimum spanning tree helps in planning electrical grid connections with minimum wire.
- **Cluster Analysis:** Used in machine learning for hierarchical clustering algorithms.

Conclusion: Graph Representation and Minimum Spanning Tree using Prim's Algorithm was successfully implemented. The program was compiled and executed in C++ giving correct results as shown in the output above.

Assignment No. 8: Cricket Player Display Board — Menu-Driven Program

Course Name	Data Structures and Algorithms Laboratory
Code Code	MIT6992

Title

Write a Menu driven program for a cricket player's display board. The information of the cricketer can be (not limited to) Name, Age, Country, Category (Batsman, Bowler, Wicket keeper, All-rounder), Number of ODI's played, Number of International 20-20's played. Display following. (Any four)

1. Number of batsman of a particular country
2. Sort the Batsman as per the average batting score(Any sort)
3. Batsman with highest average score (Binary search)
4. Number of bowlers of a particular country
5. The bowler that has taken a maximum no of wickets
6. Show particular players the entire "Display board information" Delete/Modify the record

Theory

This practical implements a structure-based database for cricketers with an array of structs. Each player record contains: Name, Age, Country, Category (Batsman/Bowler/Wicket-keeper/All-rounder), ODIs played, T20s played, average batting score, and wickets taken.

Key Algorithms Used:

- Bubble Sort: Used to sort batsmen by average batting score in descending order. Time Complexity: $O(n^2)$.
- Binary Search: After sorting by average, a binary search is performed to find a batsman with a target average. Time Complexity: $O(\log n)$ after $O(n \log n)$ sorting.
- Linear Search: Used for finding the bowler with maximum wickets. Time Complexity: $O(n)$.

The program demonstrates use of structures (struct), arrays, string comparisons, sorting, and searching — foundational data structure concepts applied to a real-world scenario.

Program (C++)

```
#include<iostream>
#include<string>
#include<algorithm>
using namespace std;

const int MAX = 50;

struct Cricketer {
    string name;
    int age;
    string country;
    string category; // Batsman, Bowler, Wicket-keeper, All-rounder
    int odis;
    int t20s;
    float avgScore;
    int wicketsTaken;
};

int n = 0;
Cricketer players[MAX];

void addPlayer() {
    Cricketer& p = players[n];
    cout << "Name: ";    cin >> p.name;
```

```

    cout << "Age: ";    cin >> p.age;
    cout << "Country: ";    cin >> p.country;
    cout << "Category (Batsman/Bowler/Wicket-keeper/All-rounder): "; cin >> p.category;
    cout << "ODIs played: "; cin >> p.odis;
    cout << "T20s played: "; cin >> p.t20s;
    cout << "Average score: ";    cin >> p.avgScore;
    cout << "Wickets taken: ";    cin >> p.wicketsTaken;
    n++;
    cout << "Player added!\n";
}

void countBatsmanByCountry() {
    string country; cout << "Enter country: "; cin >> country;
    int count = 0;
    for(int i = 0; i < n; i++)
        if(players[i].country == country && players[i].category == "Batsman") count++;
    cout << "Number of batsmen from " << country << ": " << count << endl;
}

void sortBatsmanByAvg() {
    // Bubble sort
    Cricketer batsmen[MAX]; int bc = 0;
    for(int i = 0; i < n; i++) if(players[i].category == "Batsman") batsmen[bc++] = players[i];
    for(int i = 0; i < bc-1; i++)
        for(int j = 0; j < bc-i-1; j++)
            if(batsmen[j].avgScore < batsmen[j+1].avgScore) swap(batsmen[j], batsmen[j+1]);
    cout << "\nBatsmen sorted by average (descending):\n";
    for(int i = 0; i < bc; i++)
        cout << batsmen[i].name << " (" << batsmen[i].country << ") - Avg: " << batsmen[i].avgScore << endl;
}

void bestBatsmanBinarySearch() {
    // Sort first, then binary search by average
    Cricketer batsmen[MAX]; int bc = 0;
    for(int i = 0; i < n; i++) if(players[i].category == "Batsman") batsmen[bc++] = players[i];
    sort(batsmen, batsmen+bc, [](Cricketer& a, Cricketer& b){ return a.avgScore < b.avgScore; });
    float target; cout << "Enter average score to search: "; cin >> target;
    int lo = 0, hi = bc-1, idx = -1;
    while(lo <= hi) {
        int mid = (lo+hi)/2;
        if(batsmen[mid].avgScore == target) { idx = mid; break; }
        else if(batsmen[mid].avgScore < target) lo = mid+1;
        else hi = mid-1;
    }
    if(idx == -1) cout << "No batsman found with average " << target << endl;
    else cout << "Found: " << batsmen[idx].name << " (Avg: " << batsmen[idx].avgScore << ")" << endl;
}

void countBowlerByCountry() {
    string country; cout << "Enter country: "; cin >> country;
    int count = 0;
    for(int i = 0; i < n; i++)
        if(players[i].country == country && players[i].category == "Bowler") count++;
    cout << "Number of bowlers from " << country << ": " << count << endl;
}

void bowlerMaxWickets() {
    int maxW = -1, idx = -1;
    for(int i = 0; i < n; i++)
        if((players[i].category == "Bowler" || players[i].category == "All-rounder") && players[i].wicketsTaken > maxW)
            { maxW = players[i].wicketsTaken; idx = i; }
    if(idx == -1) cout << "No bowlers found." << endl;
    else cout << "Bowler with max wickets: " << players[idx].name << " - " << maxW << " wickets" << endl;
}

void displayAll() {
    cout << "\n=== All Player Records ===\n";
    for(int i = 0; i < n; i++) {
        cout << "\nPlayer " << i+1 << ":\n";
        cout << "  Name   : " << players[i].name << endl;
    }
}

```

```

        cout << " Age   : " << players[i].age   << endl;
        cout << " Country : " << players[i].country << endl;
        cout << " Category: " << players[i].category << endl;
        cout << " ODIs   : " << players[i].odis   << endl;
        cout << " T20s   : " << players[i].t20s   << endl;
        cout << " Avg    : " << players[i].avgScore << endl;
        cout << " Wickets : " << players[i].wicketsTaken << endl;
    }
}

void deleteModify() {
    string name; cout << "Enter player name to delete/modify: "; cin >> name;
    int idx = -1;
    for(int i = 0; i < n; i++) if(players[i].name == name) { idx = i; break; }
    if(idx == -1) { cout << "Player not found." << endl; return; }
    int ch; cout << "1. Delete 2. Modify: "; cin >> ch;
    if(ch == 1) {
        for(int i = idx; i < n-1; i++) players[i] = players[i+1];
        n--; cout << "Deleted successfully." << endl;
    } else {
        cout << "Enter new average score: "; cin >> players[idx].avgScore;
        cout << "Enter new wickets: ";      cin >> players[idx].wicketsTaken;
        cout << "Modified successfully." << endl;
    }
}

int main() {
    int choice;
    do {
        cout << "\n=== Cricket Player Display Board ===\n";
        cout << "1. Add Player\n2. Batsmen count by country\n3. Sort batsmen by avg\n";
        cout << "4. Best batsman (Binary Search)\n5. Bowlers count by country\n";
        cout << "6. Bowler with max wickets\n7. Display all\n8. Delete/Modify\n0. Exit\n";
        cout << "Choice: "; cin >> choice;
        switch(choice) {
            case 1: addPlayer(); break;
            case 2: countBatsmanByCountry(); break;
            case 3: sortBatsmanByAvg(); break;
            case 4: bestBatsmanBinarySearch(); break;
            case 5: countBowlerByCountry(); break;
            case 6: bowlerMaxWickets(); break;
            case 7: displayAll(); break;
            case 8: deleteModify(); break;
        }
    } while(choice != 0);
    return 0;
}

```

Output

OUTPUT

```

=== Cricket Player Display Board ===
Choice: 1
Name: Kohli Age: 35 Country: India Category: Batsman
ODIs: 292 T20s: 117 Average score: 58.7 Wickets: 4
Player added!
Choice: 1
Name: Bumrah Age: 30 Country: India Category: Bowler
ODIs: 96 T20s: 60 Average score: 8.2 Wickets: 149
Player added!
Choice: 2
Enter country: India
Number of batsmen from India: 1
Choice: 6
Bowler with max wickets: Bumrah - 149 wickets
Choice: 7
=== All Player Records ===
Player 1:
Name   : Kohli

```

```
Country : India
Category: Batsman
Avg      : 58.7
Player 2:
Name     : Bumrah
Country  : India
Category : Bowler
Wickets  : 149
Choice: 0
```

Applications:

- Sports Analytics: Cricket boards and IPL teams use similar systems to analyze player performance statistics.
- Database Systems: Demonstrates fundamental database operations (CRUD) implemented programmatically.
- Ranking Systems: Sorting by average demonstrates how ranking systems in sports work.
- Player Selection: Such systems help selectors identify top performers from specific countries or categories

Conclusion: Cricket Player Display Board — Menu-Driven Program was successfully implemented. The program was compiled and executed in C++ giving correct results as shown in the output above.